

Kamino Unstaking Pool

Smart Contract Security Assessment

August 2025

Prepared for:

Kamino

Prepared by:

Offside Labs

Ronny Xing

Sirius Xie





Contents

1	About Offside Labs	2
2	Executive Summary	3
3	Summary of Findings	5
4	Key Findings and Recommendations	6
4.1	Missing sync_native Allows DoS on wsol_vault in collect IX	6
4.2	remaining_sol_amount Reallocation Risks Reserve Stake Withdrawal Failure .	7
4.3	Withdrawal Fees Can be Skipped	8
4.4	Incorrect Minimum Delegation Amount Leads to Possible Deposit Failure	9
4.5	Incorrect Calculation When State Account is Removable	10
4.6	Intermediate Variables Out of Sync in Simulation	11
4.7	Excessive Rigidity in Stake Pool Account Checks	13
4.8	max_sol_to_unstake Calculation Ignores Validator Removal	13
4.9	Stake Account Not Closed During Collect	14
4.10	Incorrect Validator Removal Condition	14
4.11	Missing Transient Stake Case in Calculation	15
4.12	Missing Check on unstaking_ticket_burn_authority in mint IX	16
4.13	Missing Accounting for Stake Accounts' Rent	17
4.14	Informational and Undetermined Issues	17
5	Disclaimer	21



1 About Offside Labs

Offside Labs is a leading security research team, composed of top talented hackers from both academia and industry.

We possess a wide range of expertise in modern software systems, including, but not limited to, *browsers, operating systems, IoT devices, and hypervisors*. We are also at the forefront of innovative areas like *cryptocurrencies* and *blockchain technologies*. Among our notable accomplishments are remote jailbreaks of devices such as the **iPhone** and **PlayStation 4**, and addressing critical vulnerabilities in the **Tron Network**.

Our team actively engages with and contributes to the security community. Having won and also co-organized *DEFCON CTF*, the most famous CTF competition in the Web2 era, we also triumphed in the **Paradigm CTF 2023** within the Web3 space. In addition, our efforts in responsibly disclosing numerous vulnerabilities to leading tech companies, such as *Apple, Google, and Microsoft*, have protected digital assets valued at over **\$300 million**.

In the transition towards Web3, Offside Labs has achieved remarkable success. We have earned over **\$9 million** in bug bounties, and **three** of our innovative techniques were recognized among the **top 10 blockchain hacking techniques of 2022** by the Web3 security community.



<https://offside.io/>



<https://github.com/offsidelabs>



https://twitter.com/offside_labs



2 Executive Summary

Introduction

Offside Labs completed a security audit of *UnstakingPool* smart contracts, starting on March 11th, 2025, and concluding on June 26th, 2025.

Project Overview

The Kamino Unstaking Pool enables users to unstake multiple LST accounts in a single IX. The unstaked assets are deposited into the pool's stake account, and users receive a corresponding amount of unstaking SOL. This unstaking SOL can later be burned to redeem the equivalent amount of wSOL.

Audit Scope

The assessment scope contains mainly the smart contracts of the unstaking-pool program for the *UnstakingPool* project.

The audit is based on the following specific branches and commit hashes of the codebase repositories:

- UnstakingPool
 - Codebase: <https://github.com/Kamino-Finance/unstaking-pool>
 - Commit Hash: f055567e3fea642898848ae39255af3280c4cb77
 - Branch: add-unstaking-program

We listed the files we have audited below:

- UnstakingPool
 - programs/unstaking-pool/src/handlers/handler_burn_shares.rs.rs
 - programs/unstaking-pool/src/handlers/handler_collect.rs
 - programs/unstaking-pool/src/handlers/handler_deposit_staked_sol.rs
 - programs/unstaking-pool/src/handlers/handler_init_pool.rs
 - programs/unstaking-pool/src/handlers/handler_update_pool_config.rs
 - programs/unstaking-pool/src/operations/effects.rs
 - programs/unstaking-pool/src/operations/pool_operations.rs
 - programs/unstaking-pool/src/operations/standard_stake_pool_operations.rs
 - programs/unstaking-pool/src/utils/checks.rs
 - programs/unstaking-pool/src/utils/cpi_mem.rs
 - programs/unstaking-pool/src/utils/deserialize.rs
 - programs/unstaking-pool/src/utils/macros.rs
 - programs/unstaking-pool/src/utils/pda.rs
 - programs/unstaking-pool/src/utils/shares.rs
 - programs/unstaking-pool/src/errors.rs
 - programs/unstaking-pool/src/state.rs
 - programs/unstaking-pool/src/lib.rs



- UnstakingPool PR-3
 - Commit Hash: 1967766265a2a83773beecdf155b2aa05037a8e8
- UnstakingPool PR-4
 - Commit Hash: f2755f709d5c395dd8e641034336842de6f66241
- UnstakingPool PR-5
 - Commit Hash: 5cb0faeea0a7e794b23f26c35b64c8c9d9bc13fd

Findings

The security audit revealed:

- 0 critical issue
- 0 high issue
- 5 medium issues
- 8 low issues
- 5 informational issues

Further details, including the nature of these issues and recommendations for their remediation, are detailed in the subsequent sections of this report.



3 Summary of Findings

ID	Title	Severity	Status
01	Missing sync_native Allows DoS on wsol_vault in collect IX	Medium	Fixed
02	remaining_sol_amount Reallocation Risks Reserve Stake Withdrawal Failure	Medium	Fixed
03	Withdrawal Fees Can be Skipped	Medium	Fixed
04	Incorrect Minimum Delegation Amount Leads to Possible Deposit Failure	Medium	Fixed
05	Incorrect Calculation When State Account is Removable	Medium	Fixed
06	Intermediate Variables Out of Sync in Simulation	Low	Fixed
07	Excessive Rigidity in Stake Pool Account Checks	Low	Fixed
08	max_sol_to_unstake Calculation Ignores Validator Removal	Low	Fixed
09	Stake Account Not Closed During Collect	Low	Fixed
10	Incorrect Validator Removal Condition	Low	Fixed
11	Missing Transient Stake Case in Calculation	Low	Fixed
12	Missing Check on unstaking_ticket_burn_authority in mint IX	Low	Acknowledged
13	Missing Accounting for Stake Accounts' Rent	Low	Acknowledged
14	Possible ATA Creation Failure Could Prevent Unstaking Pool Creation	Informational	Fixed
15	Inapplicability of minimum_staked_delegate Check in Specific Case	Informational	Fixed
16	Inaccurate Log	Informational	Fixed
17	Ensure action_authority Signer Is Not Duplicate	Informational	Partially Fixed
18	Issues When stake_raise_minimum_delegation_to_1_sol Feature Gate is Enabled	Informational	Acknowledged



4 Key Findings and Recommendations

4.1 Missing sync_native Allows DoS on wsol_vault in collect IX

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `collect IX`, staked SOL is withdrawn using `StakeInstruction::Withdraw` and deposited into `wsol_vault`.

```
27 let wsol_vault_balance_before =  
    amount(&ctx.accounts.wsol_vault.to_account_info())?;  
28 ...  
29 cpi_collect_sol(&ctx, wsol_to_pool, base_pool_signer_seeds)?;  
30 wrap_sol(&ctx, wsol_to_pool, base_pool_signer_seeds)?;  
31 let wsol_vault_balance_after =  
    amount(&ctx.accounts.wsol_vault.to_account_info())?;  
32 require_eq!(  
33     wsol_vault_balance_before + wsol_to_pool,  
34     wsol_vault_balance_after,  
35     UnstakingPoolError::UnexpectedSolToCollect  
36 );
```

[programs/unstaking-pool/src/handlers/handler_collect.rs#L27-L36](#)

To verify the correctness of this IX, a final check is performed to confirm that `wsol_vault` has indeed received the expected amount of SOL.

However, when obtaining the value of `wsol_vault_balance_before`, there is no prior `sync_native` operation on the `wsol_vault` account. This means that `wsol_vault_balance_before` may be less than the actual lamports in `wsol_vault`.

The subsequently obtained `wsol_vault_balance_after` value is recorded after performing `sync_native` on the `wsol_vault` account.

As a result, the final `require_eq` condition may not always hold. The actual relationship should be:

$$\text{wsol_vault_balance_before} + \text{wsol_to_pool} \leq \text{wsol_vault_balance_after}$$

Impact

A malicious user can exploit this by transferring just 1 lamport directly into the target `wsol_vault` without triggering `sync_native`, making the `require_eq` condition al-



ways fail. This prevents the `collect` operation from succeeding, effectively causing a DoS on the specific `wsol_vault` at minimal cost.

Recommendation

It is recommended to perform a `sync_native` on the `wsol_vault` account before obtaining `wsol_vault_balance_before` to ensure the `collect` IX can complete correctly.

Mitigation Review Log

Fixed in the commit `dabfef33a9ccf848895e59da82a3270ad4708740`.

4.2 `remaining_sol_amount` Reallocation Risks Reserve Stake Withdrawal Failure

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When `remaining_sol_amount < minimum_sol_delegated`, the `staked_sol_to_unstake` is reallocated to ensure that at least the minimum required amount is consumed by the last validator.

```
86     if remaining_sol_amount > 0 && remaining_sol_amount <
87         minimum_sol_delegated {
88         *staked_sol_to_unstake =
89
90         (*staked_sol_to_unstake).saturating_sub(minimum_staked_delegate);
91         *expected_sol_to_unstake =
92             utils::stake_pool_tokens_to_sol_with_fee(&stake_pool,
93                 *staked_sol_to_unstake)?;
```

[programs/unstaking-pool/src/operations/pool_operations.rs#L86-L91](#)

However, if the last `withdraw_stake_from` is `stake_pool.reserve_stake`, the amount of SOL withdrawn (`remaining_sol_amount`) in the final withdrawal can be less than `minimum_sol_delegated`.

Impact

If the final withdrawal is from `stake_pool.reserve_stake`, and the amount of SOL withdrawn in the last two withdrawal is less than `minimum_sol_delegated * 2`, the withdrawal would have succeeded originally. However, after the reallocation of



`staked_sol_to_unstake` in the code mentioned above, the second-to-last withdrawal will fail because it does not meet the `minimum_sol_delegated` requirement.

Recommendation

Check if the last withdrawal is from `stake_pool.reserve_stake` , then dont reallocate the `staked_sol_to_unstake` .

Mitigation Review Log

Fixed in the commit 25d32bd56862d5bfa59ee8f6d54ee94d684be032.

4.3 Withdrawal Fees Can be Skipped

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

According to the code of the SPL Stake Pool Program, there are two scenarios where withdrawal fees can be skipped.

```
3075     let pool_tokens_fee = if stake_pool.manager_fee_account ==  
3076         *burn_from_pool_info.key  
3077     ||  
3078     stake_pool.check_manager_fee_info(manager_fee_info).is_err()  
3079     {  
3078         0  
3079     }
```

[program/src/processor.rs#L3075-L3079](#)

1. `user_staked_sol_token` is `stake_pool.manager_fee_account` ;
2. `stake_pool.manager_fee_account` is closed or not a valid pool token account.

Impact

This would lead to inconsistencies between the simulated calculations and the actual results of the CPI , causing the post check in the DepositStakedSol instruction to fail.

Recommendation

These edge cases must be considered when calculating fees.



Mitigation Review Log

Fix in the commit `b23d01f492cdf835c946874950bad2393c749b88`.

4.4 Incorrect Minimum Delegation Amount Leads to Possible Deposit Failure

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When executing `deposit_staked_sol`, the function retrieves the minimum delegation amount of SOL using `stake::tools::get_minimum_delegation`. This value is obtained from the Stake Program and is currently set to 1 on the mainnet. The feature `stake_raise_minimum_delegation_to_1_sol` is never in effect on the mainnet.

However, in the Stake Pool, there is an additional requirement for the minimum delegation amount of SOL in a stake account, the delegation amount must not be less than the `minimum_delegation` value.

```
72 pub fn minimum_delegation(stake_program_minimum_delegation: u64) -> u64 {  
73     std::cmp::max(stake_program_minimum_delegation, MINIMUM_ACTIVE_STAKE)  
74 }
```

[program/src/lib.rs#L72-L74](#)

In the `withdraw_stake` IX, the stake account involved is also checked against this minimum delegation requirement. Based on the current mainnet configuration, this value is typically set to 1,000,000 (i.e. `MINIMUM_ACTIVE_STAKE`).

Impact

Since the Solana Stake Program and the SPL Stake Pool Program have different minimum delegation requirements, the Unstaking Pool uses a smaller value (1) compared to the Stake Pool for simulation calculations. This discrepancy indirectly leads to an overestimation of the computed `max_sol_to_unstake` value. This would lead to inconsistencies between the simulated calculations and the actual results of the CPI, causing the post check in the `DepositStakedSol` instruction to fail.

Recommendation

It is recommended to use `minimum_delegation()` to obtain the expected delegation SOL amount.



Mitigation Review Log

Fix in the commit `b23d01f492cdf835c946874950bad2393c749b88`.

4.5 Incorrect Calculation When State Account is Removable

Severity: Medium

Status: Fixed

Target: Smart Contract

Category: Math

Description

When the Stake Account is removable, the calculations in `deposit_staked_sol` are incorrect.

1. There is no need to plus `lamports_per_pool_token` when calculating `max_sol_to_unstake`, as the remaining lamports after `withdraw_stake_from` can be precisely computed to avoid precision loss.

```
114         // If validator is removable we can take all of the lamports -  
115         // what we already withdrawn  
116         withdraw_stake_from  
117             .lamports()  
118             .saturating_sub(already_unstaked_sol)  
119             + lamports_per_pool_token
```

[programs/unstaking-pool/src/operations/pool_operations.rs#L114-L118](#)

Because the additional `lamports_per_pool_token` here would actually be truncated by the stake pool program and cannot be withdrawn, it is better to calculate the precise and withdrawable `max_sol_to_unstake`.

2. Should use `expected_sol_to_unstake[current_idx] - lamports_per_pool_token` instead of `max_sol_to_unstake`:

```
143         if is_candidate_removable {  
144             // If the candidate is removable then the withdrawn amount is  
145             // only the lamports it has  
146             // but you must pass as staked sol the lamports it has + some  
147             // margin  
148             // Note that this trigger a recalculation as from now on every  
149             // candidate is removable or the reserve  
150             expected_sol_to_unstake[current_idx] =  
151                 max_sol_to_unstake.saturating_sub(lamports_per_pool_token);  
152         }
```

[programs/unstaking-pool/src/operations/pool_operations.rs#L143-L149](#)



However, due to min operations and fee deductions before this branch, `lamports_per_pool_token` cannot be directly used here to calculate the actual withdrawable SOL amount.

However, if the logic mentioned in Item 1. above is implemented, this entire branch can be completely removed.

Impact

If the simulation calculation is incorrect, it may cause the subsequent unstake operation to fail due to errors from the stake pool or stake program.

Recommendation

Here is the recommendation to fix the calculation.

The core mitigation approach involves optimizing rounding precision in the `sol_to_stake_pool_tokens_with_inverse_fee` method. Currently, the program calls `sol_to_stake_pool_tokens_with_inverse_fee` across three distinct scenarios to calculate the fee-inclusive pool tokens corresponding to the SOL to be withdrawn:

1. When calculating `minimum_staked_delegate`, it should always be rounded up; otherwise, the actual SOL obtained from unstaking may be less than `minimum_staked_delegate`. The test below demonstrates this (after applying the mitigation patch).
2. When calculating the pool tokens that need to be unstaked for a removable validator, we should always round up, but we need to limit the maximum value when calculating `expected_sol_to_unstake`. This is because the maximum value here is constrained by the validator's lamports to be removed.
3. In other cases, we don't care whether `sol_to_stake_pool_tokens_with_inverse_fee` uses rounding up or down. Although we can make a patch that employs a complex repetitive computation logic to ensure rounding up doesn't make unstaked sol exceed `max_sol_to_unstake`, this is completely unnecessary. In such cases, we can simply use rounding down instead.

Mitigation Review Log

Fixed in the commit `a3914a78a608062401938f3406ffe62be33636eb`.

4.6 Intermediate Variables Out of Sync in Simulation

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error



Description

During the simulation of the `deposit_staked_sol` function, some intermediate variables are not updated to synchronize their state after each `expected_sol_to_unstake` simulation calculation. This could lead to inconsistencies between the simulated results and the actual outcomes of the CPI execution, causing the instruction to fail.

Due to precision remnants during pool token burn, the rate of the stake pool may increase after every stake withdrawal, leading to inaccurate calculations of the following intermediate variables:

1. `minimum_staked_delegate` : This value is calculated only once before entering the simulation loop, based on the initial values of `stake_pool`.
2. `remaining_sol_amount` : When calculating `remaining_sol_amount`, `stake_pool.total_lamports` and `stake_pool.pool_token_supply` are not updated.

Additionally, the lamports in `withdraw_stake_from` are not updated after each simulation, which leads to incorrect simulation results even when the duplicate `withdraw_stake_from` address is provided repeatedly (this is a user input error, not a bug of the program).

Impact

The simulation results might not align with the actual execution due to precision issues in the simulation.

Recommendation

After each `expected_sol_to_unstake` calculation, update the `stake_pool` and `minimum_staked_delegate`. If `expected_sol_to_unstake` needs to be recalculated within the same loop, these variables should be rolled back.

Mitigation Review Log

Kamino Team: Fixed in the commit `b23d01f492cdf835c946874950bad2393c749b88`.

Offside Labs: For the newly introduced function `precompute_remaining_candidates_removable_or_reserve_vector` in the fix, its parameter `empty_stake_account_upper_bound_lamports` is derived from `stake_pool.get_lamports_per_pool_token()`. Within the function's loop, there are repeated comparisons between `stake_account.lamports` and `empty_stake_account_upper_bound_lamports`. However, a minor issue exists here: during each iteration, after withdrawing a stake from the stake pool, the value of `stake_pool.get_lamports_per_pool_token()` changes. Theoretically, this means `empty_stake_account_upper_bound_lamports` could also vary in each iteration. In practice, for most stake pools, `lamports_per_pool_token` is typically 2, and the likelihood of it changing significantly is quite low.



4.7 Excessive Rigidity in Stake Pool Account Checks

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Data Validation

Description

The SPL Stake Pool program does not strictly limit the size of stake pool accounts.

However, the `validate_standard_stake_pool` function strictly validates that the size of the stake pool account must be `STAKE_POOL_SIZE`. This could result in some valid stake pools being rejected.

Recommendation

Validate the stake pool according to the approach of SPL Stake Pool program

1. owner is `spl_stake_pool::ID`;
2. `data_len` $\geq$ `STAKE_POOL_SIZE` ;
3. `account_type == AccountType::StakePool`

Mitigation Review Log

Fixed in the commit `b23d01f492cdf835c946874950bad2393c749b88`.

4.8 `max_sol_to_unstake` Calculation Ignores Validator Removal

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

If the amount of SOL being withdrawn is exactly $\geq$ all the lamports in the validator stake account and $\leq$ this lamports + lamports_per_pool_token, then it is possible to withdraw all the lamports from that stake account of the validator and remove the validator.

Impact

The `max_sol_to_unstake` in the `deposit_staked_sol` method does not consider this scenario, which could prevent the staked SOL that should be withdrawn from being extracted properly in some cases.



Recommendation

Following the above withdraw lamports check to calculate the `max_sol_to_unstake` for validator removal.

Mitigation Review Log

Fixed in the commit 25d32bd56862d5bfa59ee8f6d54ee94d684be032.

4.9 Stake Account Not Closed During Collect

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

In the `CollectSol` instruction, when invoking the `StakeInstruction::Withdraw` CPI, the `wsol_to_pool` parameter does not include the rent for the stake account. This prevents the stake account from being closed and leaves the rent permanently locked in the account, unable to be withdrawn.

Recommendation

This rent comes from the user who deposited the staked SOL, but to avoid adding extra accounting overhead, it's recommended to directly collect this rent and transfer it to the admin address or the payer who is calling the `CollectSol` instruction.

Mitigation Review Log

Offside Labs: If a stake account has neither active stake nor is in the deactivation status, it should be possible to withdraw all lamports from it using `StakeInstruction::Withdraw`. This should also include the rent in the stake account. So, closing a stake account shouldn't be limited to just merging it with another stake account.

Kamino Team: Fixed in the commit b55883d6378cd49537cad066a30d4b0a7e3d2288.

4.10 Incorrect Validator Removal Condition

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error



Description

In the call to `precompute_remaining_candidates_removable_or_reserve_vector`, the condition used to determine whether a stake account is considered empty (`StakeWithdrawSource::ValidatorRemoval`) is: `candidates[current_idx].lamports() <= minimum_sol_delegated + lamports_per_pool_token`. However, the correct condition should be: `candidates[current_idx].lamports() <= minimum_sol_delegated + lamports_per_pool_token + STAKE_ACCOUNT_RENT_EXEMPTION`.

Impact

This may cause validators with balances within this range to be missed and not marked for removal.

Recommendation

It is recommended to update the condition to: `candidates[current_idx].lamports() <= minimum_sol_delegated + lamports_per_pool_token + STAKE_ACCOUNT_RENT_EXEMPTION`

Mitigation Review Log

Fixed in the commit `b55883d6378cd49537cad066a30d4b0a7e3d2288`.

4.11 Missing Transient Stake Case in Calculation

Severity: Low

Status: Fixed

Target: Smart Contract

Category: Logic Error

Description

When the remaining lamports in a stake account fall within the range `[minimum_sol_delegated + 1, minimum_sol_delegated + lamports_per_pool_token]`, the `withdraw_stake` logic will classify this case as `StakeWithdrawSource::ValidatorRemoval`, and forcibly set `withdraw_lamports = stake_account.lamports()` ([program/src/processor.rs#L2911-L2927](#)). However, in the unstaking pool, the `max_sol_to_unstake` is calculated as `minimum_sol_delegated + STAKE_ACCOUNT_RENT_EXEMPTION`, which ignores the possible extra `[1, lamports_per_pool_token]` lamports.

Impact

This may lead to inconsistencies between the simulation and actual CPI execution. Furthermore, if `unstake_sol < minimum_sol_delegated + STAKE_ACCOUNT_RENT_EXEMPTION +`



1, the following CPI call to withdraw staked SOL might fail here: [program/src/processor.rs#L2914](#)

Recommendation

It's recommended to adjust the calculation logic to handle this edge case properly.

Mitigation Review Log

Fixed in the commit [25d32bd56862d5bfa59ee8f6d54ee94d684be032](#).

4.12 Missing Check on unstaking_ticket_burn_authority in mint IX

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Data Validation

Description

In the `mint` IX, an `UnstakingTicket` is created, and `DepositStakedSol.unstaking_ticket_burn_authority` is updated into `UnstakingTicket.burn_authority`.

However, there is no requirement that `unstaking_ticket_burn_authority` must be the signer of the mint IX, i.e., the user.

```
164 pub unstaking_ticket_burn_authority: AccountInfo<'info>,  
programs/unstaking-pool/src/handlers/handler\_deposit\_staked\_sol.rs#L164-L164
```

Meanwhile, in the `burn` IX, it is explicitly required that `UnstakingTicket.burn_authority` must be the user.

```
102 #[account(mut,  
103     constraint = unstake_ticket.load()?.burn_authority == user.key()  
104     @ UnstakingPoolError::WrongBurnAuthority,  
105     )]  
105 pub unstake_ticket: AccountLoader<'info, UnstakeTicket>,  
programs/unstaking-pool/src/handlers/handler\_burn\_shares.rs#L102-L105
```

Impact

If User A passes in an User B's account as the `burn_authority` in the mint IX, then the USOL minted by A cannot be burned by A unless B voluntarily updates the `burn_authority` to A using the `change_ticket_authority` IX.

Even if B owns the ticket originally created by A, B may still not be able to burn the corresponding USOL to receive WSOL, because B's USOL token account may not contain the



amount recorded in the ticket. This would cause the burn to fail due to an error in the `token::burn` IX.

Recommendation

It is recommended to add a constraint in the `mint` IX to ensure `unstaking_ticket_burn_authority == user`.

Alternatively, if this is by design, the mint IX could directly mint USOL to the USOL token account associated with `unstaking_ticket_burn_authority`.

4.13 Missing Accounting for Stake Accounts' Rent

Severity: Low

Status: Acknowledged

Target: Smart Contract

Category: Logic Error

Description

The `collect` instruction withdraws and records the stake account's rent to `pool.wsol_in_vault`. However, this portion of SOL is not included in `pool.shares_issued`, making it impossible to withdraw.

Impact

Users cannot withdraw the rent paid during deposit.

Recommendation

It is recommended to add separate accounting and an admin instruction specifically for collecting these rent SOL.

4.14 Informational and Undetermined Issues

Possible ATA Creation Failure Could Prevent Unstaking Pool Creation

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

In the `init_pool` IX, the `wsol_vault` is initialized. However, a malicious user can front-run the `init_pool` IX by creating the `wsol_vault` ATA beforehand. This would cause the `init_pool` IX to fail when attempting to create the `wsol_vault` account, preventing



the corresponding unstaking pool from being created. It is recommended to change `init` to `init_if_needed` in the Anchor constraints.

Inapplicability of `minimum_staked_delegate` Check in Specific Case

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

At the beginning of the `deposit_staked_sol` method, there is a minimum value check for the total `staked_sol_to_deposit`, requiring that `staked_sol_to_deposit` must be `>= minimum_staked_delegate`. However, if the stake is directly withdrawn from the pool reserve, this restriction should not apply.

On the other hand, a reasonable minimum limit helps avoid excessive precision loss, so retaining this check is also justified. It's best to clarify this in the code comments.

Inaccurate Log

Severity: Informational

Status: Fixed

Target: Smart Contract

Category: Logic Error

If the number of `withdraw_stake_from` is insufficient, `staked_sol_to_deposit` may not be fully consumed. The log printed here is inaccurate.

```
56 msg!(
57     "staked_sol_to_deposit {} and shares_to_mint {}",
58     staked_sol_to_deposit,
59     shares_to_mint
60 );
```

[programs/unstaking-pool/src/handlers/handler_deposit_staked_sol.rs#L56-L60](#)

Ensure `action_authority` Signer Is Not Duplicate

Severity: Informational

Status: Partially Fixed

Target: Smart Contract

Category: Data Validation

Recommended to check that `action_authority != user` to prevent attackers from consuming stake pool tokens owned by the `action_authority`.

And also recommended to verify that `action_authority` is not same as any of `withdraw_stake_to` to prevent attackers from initializing `action_authority` as a stake account, which would render `action_authority` unusable as the fee payer.

Issues When `stake_raise_minimum_delegation_to_1_sol` Feature Gate is Enabled

Severity: Informational

Status: Acknowledged

Target: Smart Contract

Category: Logic Error



The minimum staking delegation requirement on the mainnet is currently set at 1 lamport. There is a pending feature gate `stake_raise_minimum_delegation_to_1_sol`, which will raise this minimum delegation to 1e9 lamports.

This feature is currently unimplemented and may never be enabled, as it would break the existing stake pool program.

The following issues(also are the issues mentioned in the above *Note* callout) only manifests when the feature is enabled, and may cause the DepositStakedSol instruction to fail in edge cases.

1. If the `remaining_sol_amount` for the last `withdraw_stake_from` account is less than `minimum_sol_delegated` (1e9 lamports).
2. If the `max_sol_to_unstake` for a `withdraw_stake_from` account is less than `minimum_sol_delegated` (1e9 lamports).
This is a bug for SPL Stake Pool program if the `stake_raise_minimum_delegation_to_1_sol` feature is enabled.

3. If a `withdraw_stake_from` account meets this branch's conditions,

```
169     if remaining_sol_amount > 0
170         && remaining_sol_amount < minimum_sol_delegated
171         && expected_sol_to_unstake[current_idx] >= 2 *
    => minimum_sol_delegated
172         && current_idx + 1 <
    => stake_pool_accounts.withdraw_stake_from.len()
173         && !are_remaining_candidates_removable_or_reserve[current_idx +
    => 1]
174         .is_removable_or_reserve
```

[programs/unstaking-pool/src/operations/pool_operations.rs#L169-L174](#)

rounding precision errors in `minimum_staked_delegate` (due to ceiling) may result in `expected_sol_to_unstake` being smaller than `minimum_staked_delegate` after subtracting `minimum_staked_delegate` from `staked_sol_to_unstake`:

```
176     *staked_sol_to_unstake =
177     => (*staked_sol_to_unstake).saturating_sub(minimum_staked_delegate);
```

[programs/unstaking-pool/src/operations/pool_operations.rs#L176-L177](#)

To avoid these edge cases, we recommend:

1. Filter out candidates with lamports $\leq \text{rent} + 2 * \text{minimum_sol_delegated}$ in the off-chain SDK.
2. When either `remaining_sol_amount` or `max_sol_to_unstake` is less than `minimum_sol_delegated`, skip the `withdraw_stake_from` account directly.



3. If the above 2. is to be implemented, the calculation of slippage should be optimized. The slippage for `min_shares_to_receive` should be computed based on the ratio of `staked_sol_to_unstake` (returned by `pool_operations::deposit_staked_sol`) to the input parameter `staked_sol_to_deposit`.

Note that it's difficult to completely eliminate all edge cases in on-chain programs. For example, if a source stake account's lamports is approximately equal to or less than `minimum_required_lamports + minimum_sol_delegated` (i.e., `rent + 2 * minimum_sol_delegated`), withdrawing `max_sol_to_unstake` from this stake account may still fail (potentially due to precision errors in the 3. case mentioned above, or because `max_sol_to_unstake` has become less than `minimum_sol_delegated`). Therefore, it is recommended to mitigate these edge cases as much as possible in the off-chain SDK rather than attempting to handle all edge cases entirely on-chain.



5 Disclaimer

This audit report is provided for informational purposes only and is not intended to be used as investment advice. While we strive to thoroughly review and analyze the smart contracts in question, we must clarify that our services do not encompass an exhaustive security examination. Our audit aims to identify potential security vulnerabilities to the best of our ability, but it does not serve as a guarantee that the smart contracts are completely free from security risks.

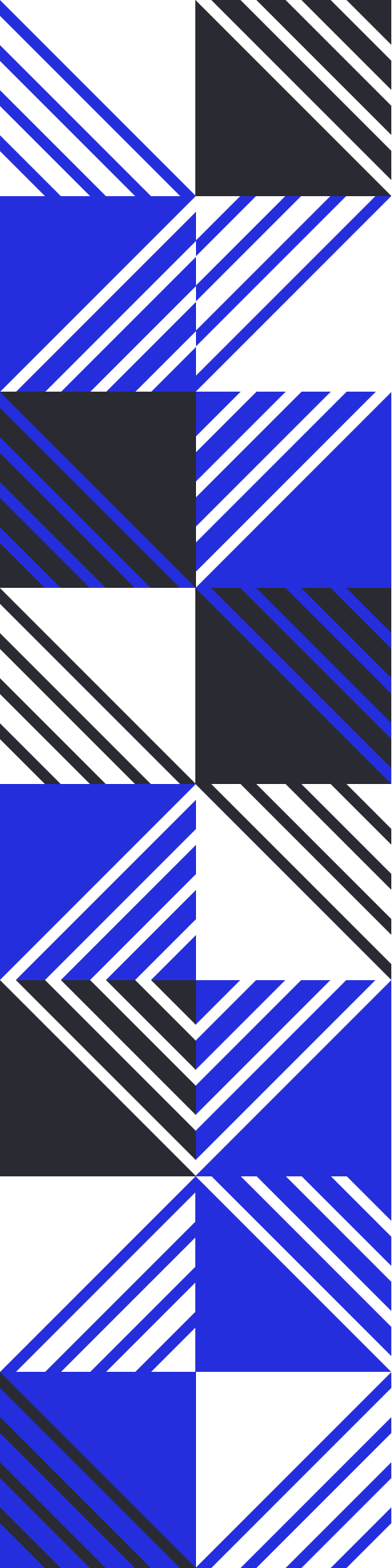
We expressly disclaim any liability for any losses or damages arising from the use of this report or from any security breaches that may occur in the future. We also recommend that our clients engage in multiple independent audits and establish a public bug bounty program as additional measures to bolster the security of their smart contracts.

It is important to note that the scope of our audit is limited to the areas outlined within our engagement and does not include every possible risk or vulnerability. Continuous security practices, including regular audits and monitoring, are essential for maintaining the security of smart contracts over time.

Please note: we are not liable for any security issues stemming from developer errors or misconfigurations at the time of contract deployment; we do not assume responsibility for any centralized governance risks within the project; we are not accountable for any impact on the project's security or availability due to significant damage to the underlying blockchain infrastructure.

By using this report, the client acknowledges the inherent limitations of the audit process and agrees that our firm shall not be held liable for any incidents that may occur subsequent to our engagement.

This report is considered null and void if the report (or any portion thereof) is altered in any manner.



 <https://offside.io/>

 <https://github.com/offsidelabs>

 https://twitter.com/offside_labs